



University of Sarajevo
Faculty of Electrical Engineering
Department of Computing and Informatics



Procure Hub

Source It Right

Software requirements specification

RESPONSIBLE PROFESSOR:
Prof. Dr. Novica Nosović

RESPONSIBLE ASSISTANTS:
Mr. Dipl. Eng. Kenan Halilović
Mr. Dipl. Eng. Tarik Hrnjić

Sarajevo, June 2025.

Content

Content.....	1
1. Introduction.....	3
1.1 Purpose.....	3
1.2 Document Conventions.....	3
1.3 Intended Audience.....	4
1.4 Project Scope.....	4
1.5 References.....	5
2. System Overview.....	5
2.1 Product Perspective & Entity Relationship Diagram.....	5
2.2 Activity and Use Case Diagrams.....	7
3. Operational Environment.....	14
4. Design and Implementation Constraints.....	15
5. System Features.....	16
5.1 Description and Priority.....	16
5.2 Functional Requirements and Component Diagram.....	18
6. External Interface Requirements.....	21
6.1 User Interface.....	21
6.2 Hardware Interface.....	21
6.3 Software Interface.....	22
Backend APIs:.....	22
Caching:.....	22
Third-party and Internal Services:.....	22
Database:.....	23
6.4 Communication Interface and Deployment Diagram.....	23
Communication Interfaces:.....	23
7. Non-Functional Requirements.....	24
7.1 System Usability.....	24
7.2 System Security.....	24
7.3 System Availability.....	25
7.4 System Maintainability.....	25
7.5 System Scalability.....	25
7.6 System Performance.....	26
8. FURPS+ Model.....	26
8.1 Functionality.....	26
8.2 Usability.....	27
8.3 Reliability.....	27
8.4 Performance.....	27

8.5 Supportability.....	28
9. Implementation Requirements.....	29
9.1 Interface Constraints.....	29
9.2 Physical Requirements.....	29
9.3 Design Constraints.....	29

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and nonfunctional requirements of **ProcureHub**, a web-based procurement and auction platform. This document serves as a comprehensive guide for developers, designers, testers, and stakeholders by providing a detailed specification of the system's expected behavior, features, and constraints. The platform aims to streamline the procurement process for organizations and individuals by enabling efficient bidding, proposal submissions, live auctioning, and contract awarding through a centralized and accessible system.

1.2 Document Conventions

This document follows the IEEE standards for technical documentation to ensure clarity and consistency. The following conventions are used throughout:

- **Terminology:** Key terms and system components are capitalized (e.g., *ProcureHub*, *Buyer*, *Auction*).
- **Text Formatting:**
 - **Bold** text indicates important concepts and terms.
 - *Italic* text is used for emphasis and examples.
 - Monospace font is used for code snippets, commands, file names, and API endpoints.
- **Lists:** Bulleted and numbered lists organize information for readability.
- **Headings:** Section and subsection headings are numbered according to IEEE style.
- **Diagrams:** All diagrams follow UML notation conventions, with appropriate titles and captions.
- **References:** Citations and external references are provided in IEEE citation format.

1.3 Intended Audience

This document is intended for:

- **Project Stakeholders** – To understand system goals and requirements.
- **Developers & Engineers** – To guide implementation of features and architecture.
- **Testers & QA** – To define the basis for test plans and validation criteria.
- **UI/UX Designers** – To align design decisions with usability and functional expectations.
- **Product Owners & Managers** – To monitor scope and ensure delivery alignment.

1.4 Project Scope

ProcureHub will provide a secure, scalable, and user-friendly web platform that supports:

- Creation and management of procurement requests.
- Competitive bidding and proposal submissions.
- Real-time live auctions.
- Transparent contract awarding with document handling.
- Full role-based access for buyers, sellers, and administrators.
- Administrative oversight, compliance monitoring, and analytical dashboards.

The platform will support both small-scale and enterprise-level procurement needs, offering automation, traceability, and competitive mechanisms that reduce time, cost, and manual effort involved in traditional procurement processes

1.5 References

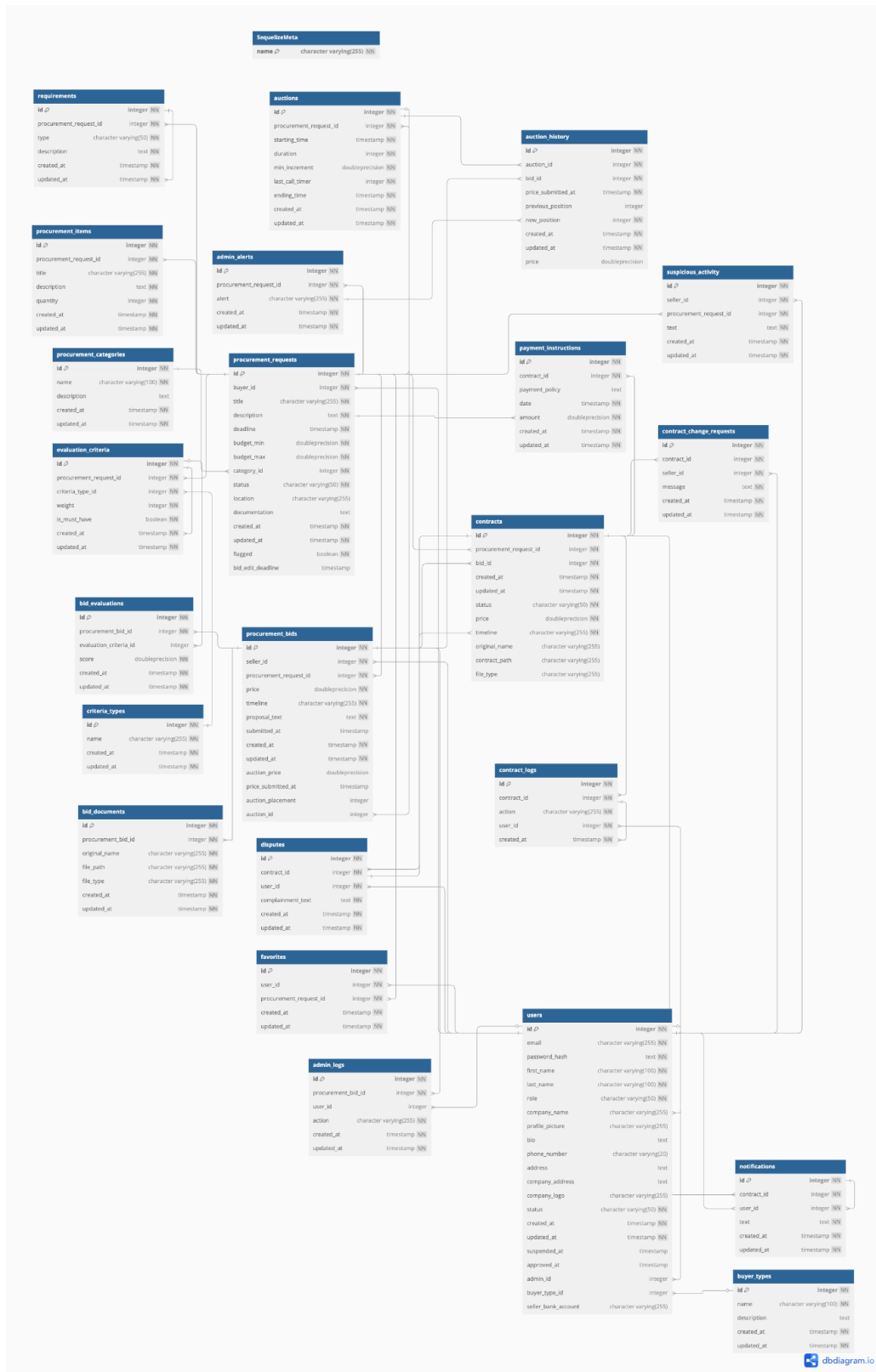
- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications
- OWASP Web Security Guidelines – <https://owasp.org>
- PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- Node.js API Reference – <https://nodejs.org/api>
- React Documentation – <https://reactjs.org>

2. System Overview

2.1 Product Perspective & Entity Relationship Diagram

ProcureHub is an independent, web-based platform built to streamline procurement and auction processes between buyers and sellers. The platform is being developed as a standalone full-stack application, designed to operate on standard web browsers without requiring additional installations. It will incorporate real-time features using WebSockets.

The system will follow a **modular architecture** with clear separation between the frontend (React.js), backend (Node.js + Express.js), and the database (PostgreSQL). The architecture will also support scalable services and real-time updates, particularly for the auction module.



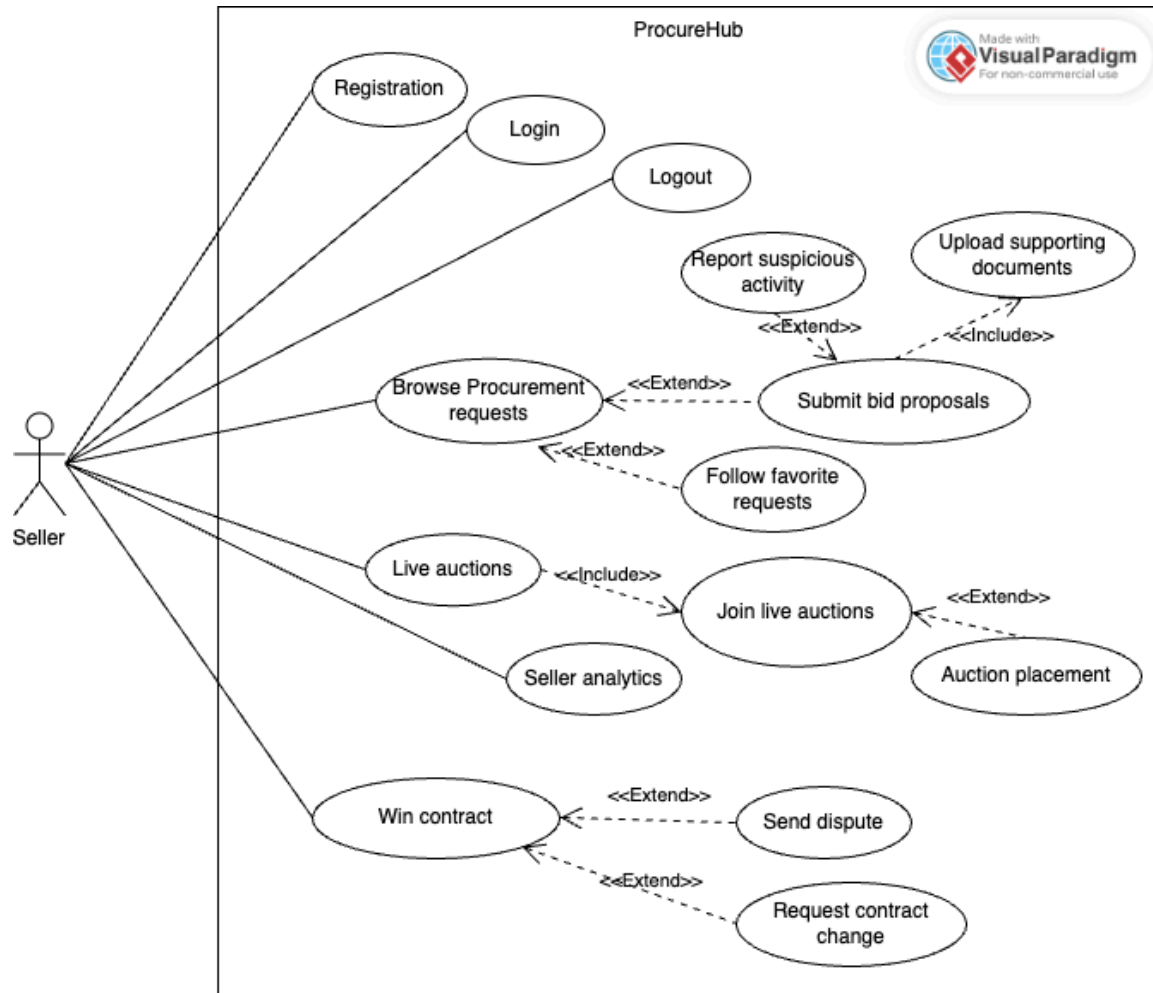
Picture 1: ERD

2.2 Activity and Use Case Diagrams

Users can have one of three roles: admin, seller and buyer. Depending on the role, the functionalities are as follows.

Seller:

- Registration
- Login
- Logout
- Browse Procurement requests
- Save favorite requests
- Submit bid proposals
- Upload supporting documents
- Report suspicious activity
- Live auctions
- Join live auctions
- Auction placement
- Seller analytics
- Win contract
- Send dispute
- Request contract change

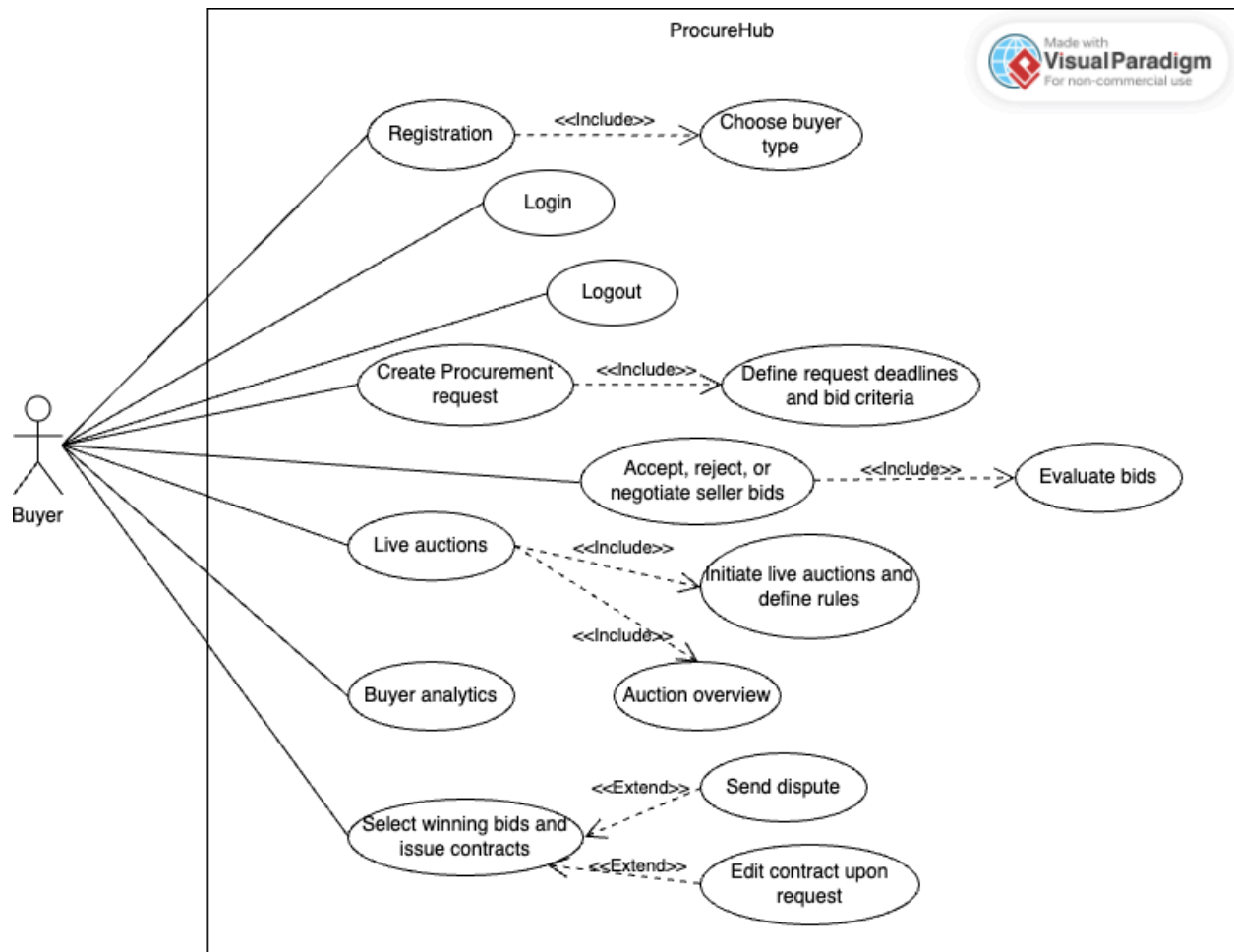


Picture 2: Use case for Seller

Buyer:

- Registration
- Choose buyer type
- Login
- Logout
- Create Procurement request
- Define request deadlines and bid criteria
- Accept, reject, or negotiate seller bids
- Evaluate bids
- Live auctions
- Initiate live auctions and define rules
- Auction overview
- Buyer analytics

- Select winning bids and issue contracts
- Send dispute
- Edit contract upon request

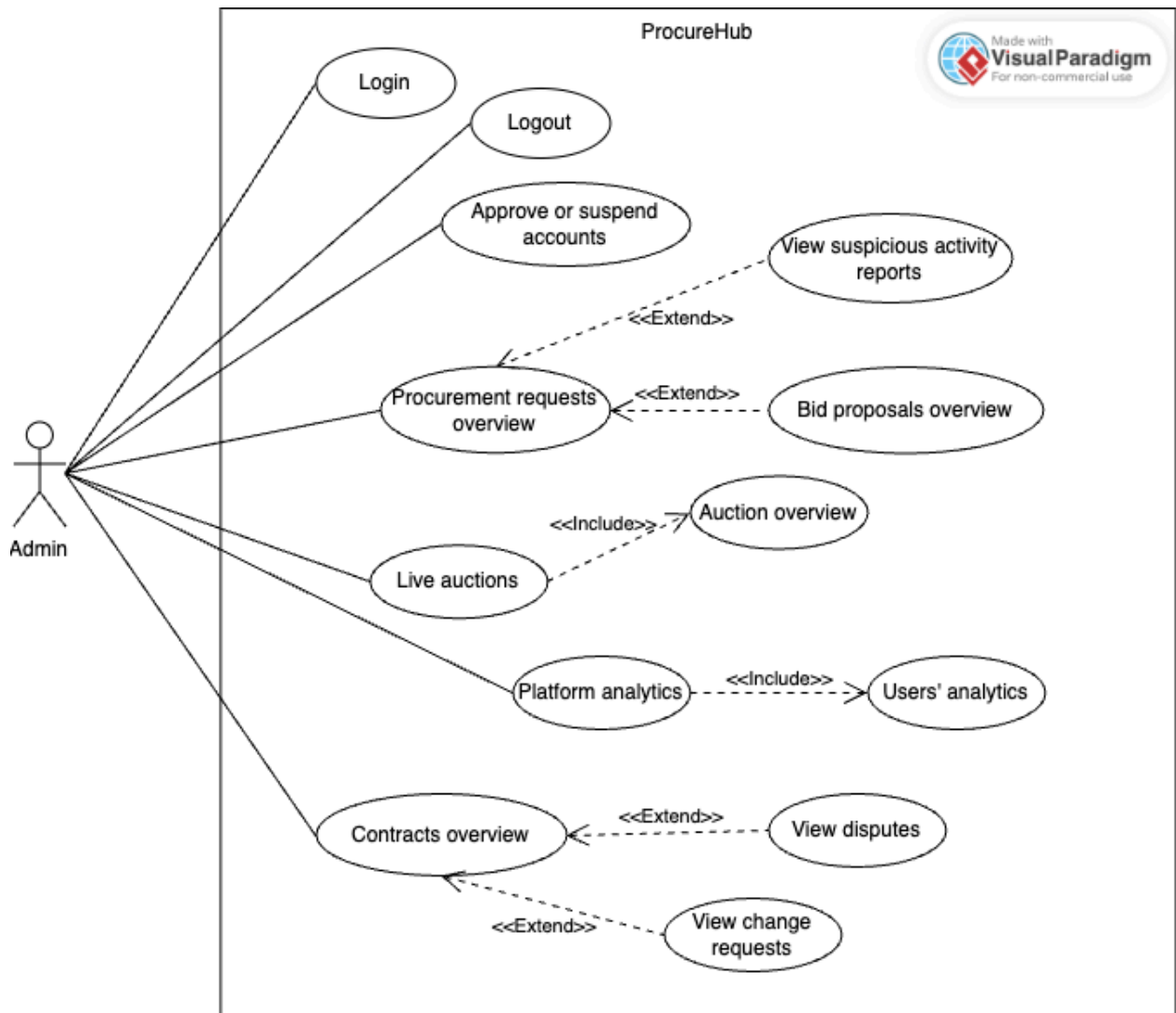


Picture 3: Use case for Buyer

Admin:

- Login
- Logout
- Approve or suspend accounts
- View suspicious activity reports
- Procurement requests overview
- Bid proposals overview
- Auction overview
- Live auctions

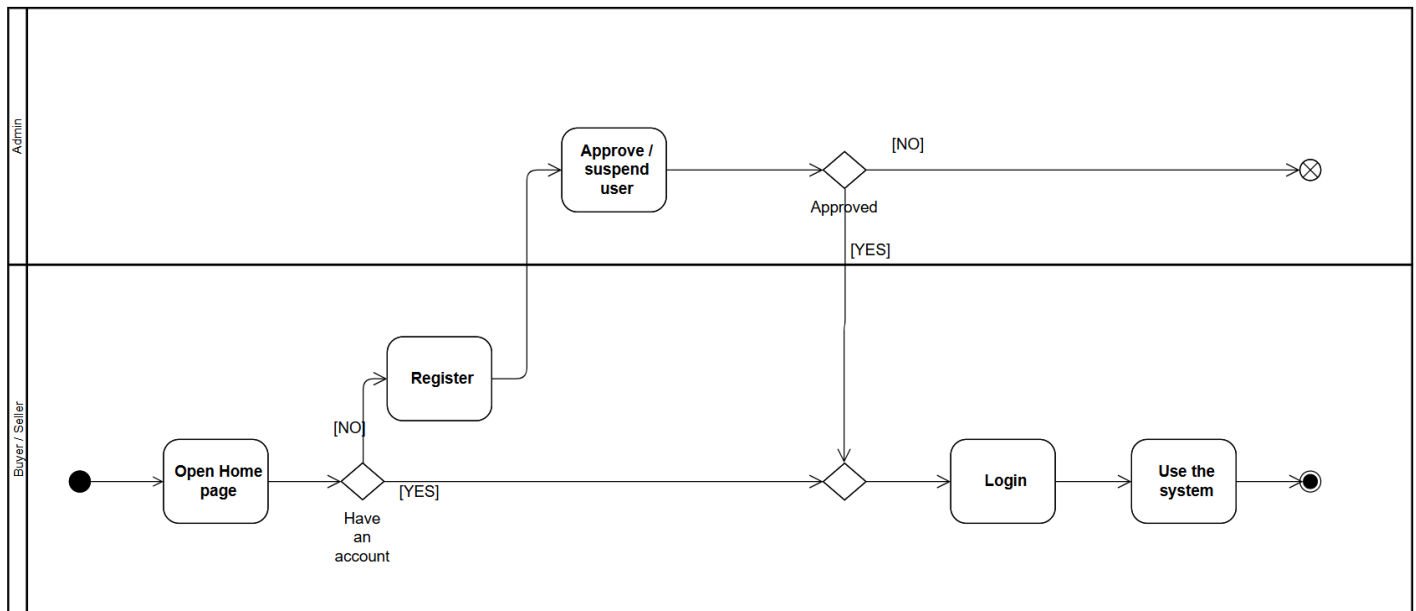
- Platform analytics
- Users' analytics
- Contracts overview
- View disputes
- View change requests



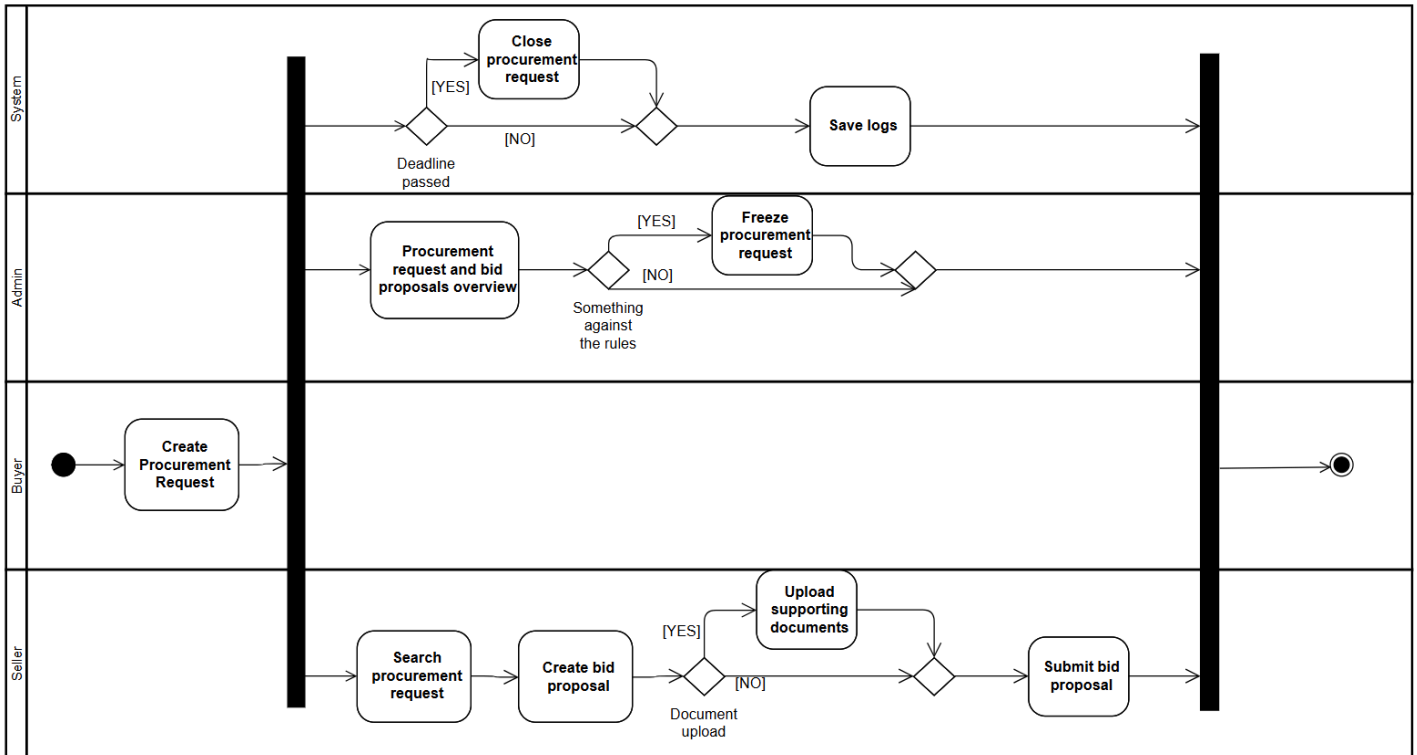
Picture 4: Use case for Admin

The most important processes will be presented through activity diagrams:

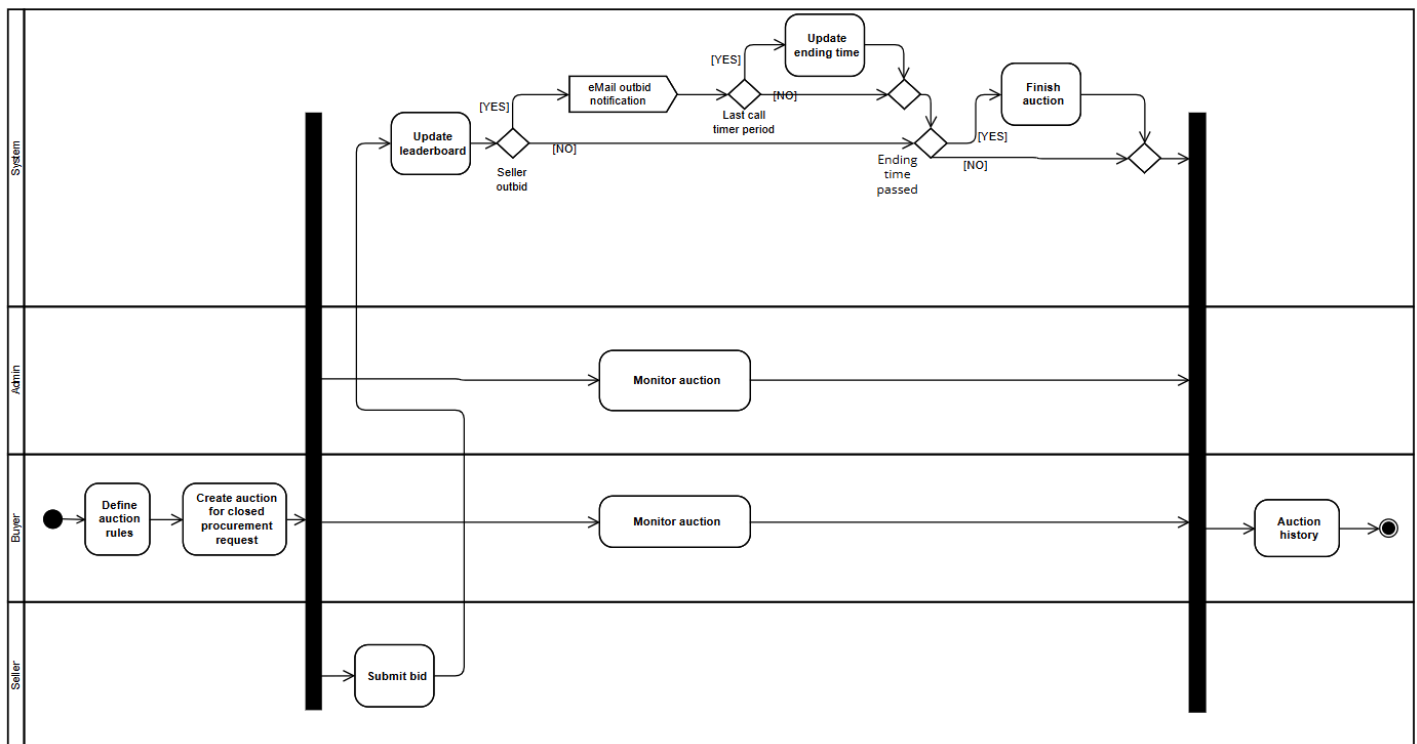
- Registration and login (Picture 5)
- Procurement request creation and bid proposal submission (Picture 6)
- Live auction (Picture 7)
- Contract issuance and acceptance (Pictures 8 and 9)



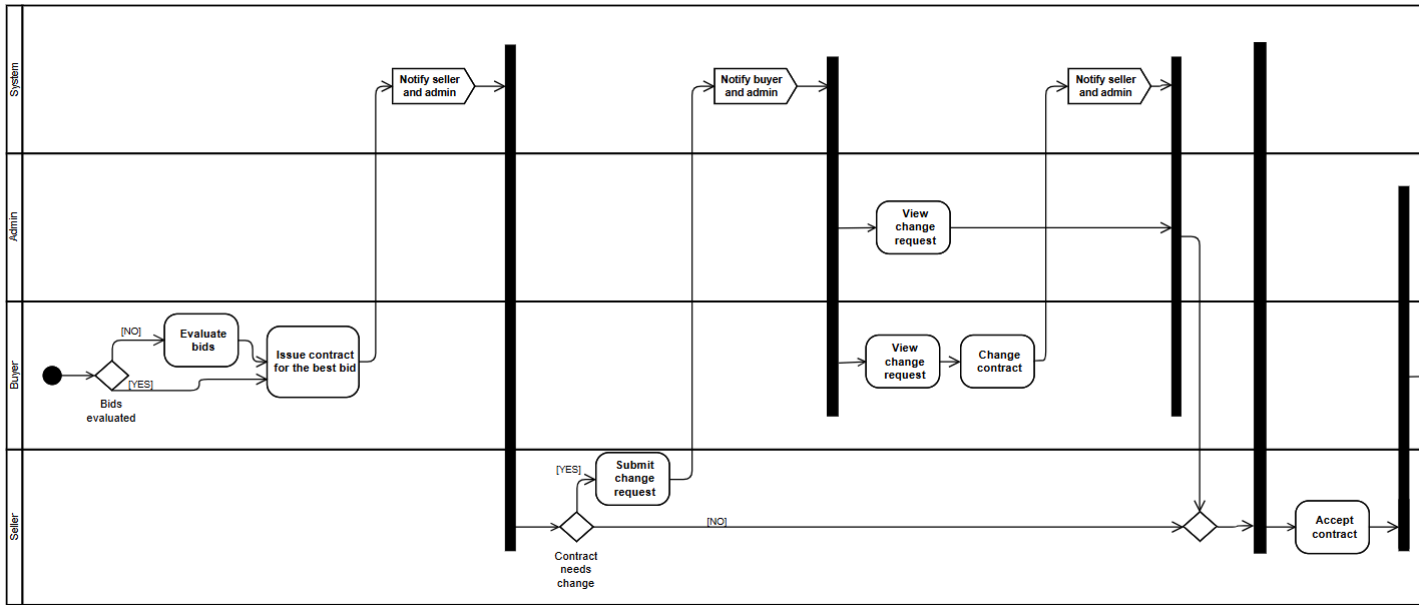
Picture 5: Activity diagram - Registration and login



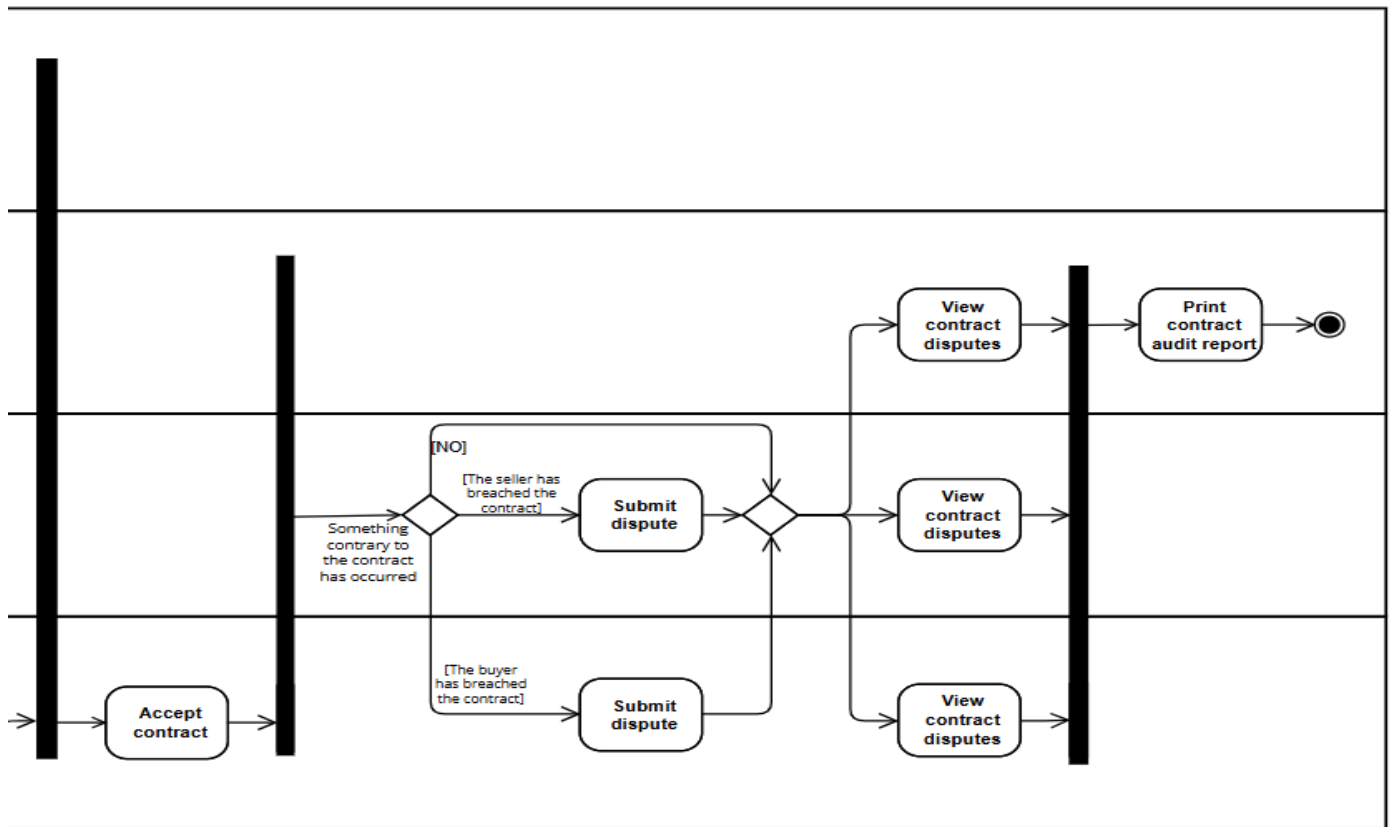
Picture 6: Activity diagram - Procurement request creation and bid proposal submission



Picture 7: Activity diagram - Live auction



Picture 8: Activity diagram - Contract issuance and acceptance (part 1)



Picture 9: Activity diagram - Contract issuance and acceptance (part 2)

3. Operational Environment

ProcureHub is implemented using a client-server architecture to support a shared centralized database. All data processing, searching, and storage of information and settings are performed on the server side.

The client application is designed to be platform-independent and accessible via modern web browsers on multiple operating systems, including:

- Windows
- Linux
- macOS

Supported web browsers with minimum versions include:

- Google Chrome (latest versions)
- Microsoft Edge 20+
- Mozilla Firefox 70+
- Opera 15+
- Apple Safari 6+

The platform is optimized for responsiveness and compatibility across devices such as desktops, laptops, and tablets.

4. Design and Implementation Constraints

- The frontend is implemented using React, chosen for its efficiency in building dynamic and responsive user interfaces.
- The backend is developed with Node.js and Express.js to provide scalable and maintainable server-side functionality.
- Authentication and authorization utilize JWT (JSON Web Tokens) for secure, stateless user sessions.
- The application must be fully functional and accessible on Windows, macOS, and Linux operating systems via supported browsers.
- The system uses a shared database architecture, where all key data persistence and business logic execute on the server.
- User roles and permissions impose access restrictions; certain features are accessible only to users with specific roles (e.g., Admin, Seller, Buyer).
- The platform supports file uploads with restrictions on size and file types to ensure security and compliance.
- Real-time functionalities such as live auctions rely on WebSocket communication protocols and must handle concurrency gracefully.
- Deployment environments require secure HTTPS communication and integration with third-party services for notifications and payments.
- The system enforces strict validation rules on inputs both client-side and server-side to maintain data integrity.

5. System Features

This section provides a detailed overview of ProcureHub's system features, outlining their descriptions, priorities, and corresponding functional requirements. It also includes an explanation of how the provided component diagram visually represents the system's architecture and the relationships between its core components.

5.1 Description and Priority

Each core feature of the ProcureHub system is described in terms of its purpose and the value it provides to users. We also assign a priority level to each feature to guide development efforts, indicating its importance relative to other features for the initial release and subsequent iterations.

Feature ID	Feature Name	Description	Priority (High/Medium/Low)
FEAT-001	User Authentication & Profile	Allows users to securely register, log in, manage their profiles, and ensures system security.	High
FEAT-002	Notification Management	Provides real-time and historical notifications to users regarding system events and actions.	High
FEAT-003	Procurement Request Handling	Enables users to create, browse, and manage their procurement requests, including favorites.	High

FEAT-004	Bidding and Proposal	Facilitates the creation, submission, and review of bids and proposals for procurement requests.	High
FEAT-005	Live Auction Functionality	Supports the creation and participation in real-time online auctions.	Medium
FEAT-006	Contract & Award Management	Manages the lifecycle of contracts, from document upload and signing to award tracking.	High
FEAT-007	Admin Dashboard & Management	Provides administrators with tools for user management, compliance monitoring, and system overviews.	High
FEAT-008	Analytics and Reporting	Integrates with an external analytics system to provide insights and generate procurement overviews.	Medium
FEAT-009	Document Management	Supports uploading and linking documents to various procurement processes (e.g., bids, contracts).	High

5.2 Functional Requirements and Component Diagram

This subsection outlines the specific functional requirements derived from the identified system features of ProcureHub. Functional requirements define what the system *must do* to satisfy user needs and business objectives. The subsequent component diagram visually illustrates the high-level architecture of the system, showing how different components interact to fulfill these requirements.

The following are key functional requirements for the ProcureHub system, directly informed by its components and their interactions:

- **FR-001: User Registration:** The system shall allow new users to register and create a user profile.
- **FR-002: User Authentication:** The system shall securely authenticate users via a Security Service for access to system functionalities.
- **FR-003: Profile Management:** The system shall enable authenticated users to view and modify their profile information.
- **FR-004: Procurement Request Creation:** The system shall allow buyers to create new procurement requests.
- **FR-005: Procurement Request Browsing:** The system shall enable sellers to browse and search existing procurement requests.
- **FR-006: Favorite Procurement Requests:** The system shall allow sellers to mark and manage procurement requests as favorites.
- **FR-007: Bid Submission:** The system shall allow sellers to submit bids and proposals for open procurement requests.
- **FR-008: Bid Review:** The system shall enable designated users (e.g., administrators, requesters) to review submitted bids.
- **FR-009: Live Auction Creation:** The system shall allow authorized users to create and configure live auctions.
- **FR-010: Live Auction Participation:** The system shall enable users to participate in ongoing live auctions.
- **FR-011: Real-time Notifications:** The system shall send real-time notifications to users via a Web Socket Interface for events such as new bids, auction updates, or status changes.
- **FR-012: Document Upload:** The system shall allow users to upload relevant documents (e.g., bid documents, contract drafts) at various stages of the procurement process.
- **FR-013: Contract Signing Tracking:** The system shall track the status of contract signing processes.
- **FR-014: Award Tracking:** The system shall enable tracking of awarded contracts.

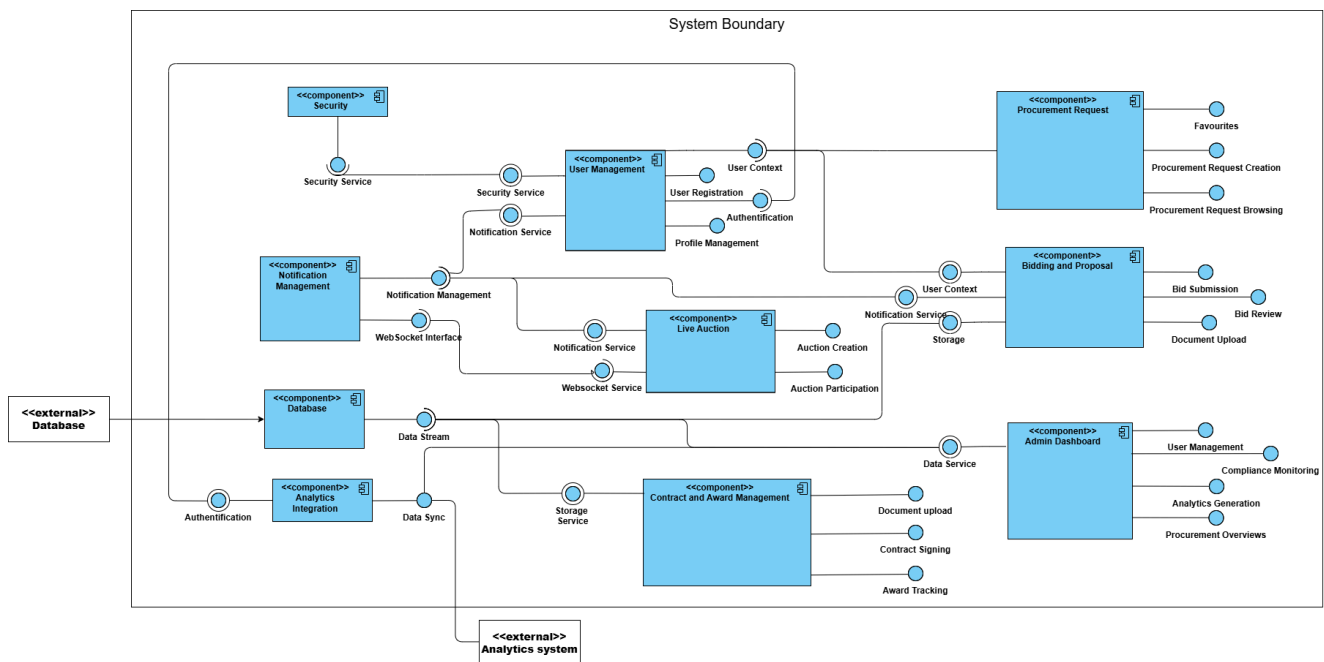
- **FR-015: User Management (Admin):** The Admin Dashboard shall allow administrators to manage user accounts (e.g., create, modify, deactivate).
- **FR-016: Compliance Monitoring:** The Admin Dashboard shall provide tools for monitoring system compliance with procurement policies.
- **FR-017: Procurement Overviews (Admin):** The Admin Dashboard shall display high-level overviews of procurement activities and statuses.
- **FR-018: Analytics Generation:** The system shall integrate with an external analytics system to generate comprehensive analytics and reports.
- **FR-019: Data Persistence:** All system data shall be securely stored and retrieved from the external Database.

The component diagram shows the following key components and their relationships:

- **System Boundary:** Encompasses all internal components of the ProcureHub application.
- **Security Component:** Provides core Security Service for authentication and authorization across the system. It interacts with the User Management component.
- **User Management Component:** Handles User Registration, Authentication (via Security Service), and Profile Management. It also serves the User Context to other components.
- **Notification Management Component:** Responsible for Notification Management and exposes a Web Socket Interface. It interacts with the Notification Service which is utilized by other components like Live Auction and Bidding and Proposal.
- **Procurement Request Component:** Manages Procurement Request Creation, Procurement Request Browsing, and Favorites. It likely interacts with the Database for data storage.
- **Live Auction Component:** Enables Auction Creation and Auction Participation, utilizing the Notification Service and Storage service.
- **Bidding and Proposal Component:** Facilitates Bid Submission, Bid Review, and Document Upload. It also uses the Notification Service and Storage service.
- **Contract and Award Management Component:** Oversees Document upload, Contract Signing, and Award Tracking. This component interacts with the Data Service and likely the Database.
- **Admin Dashboard Component:** Provides administrative functionalities including User Management, Compliance Monitoring, Analytics Generation, and Procurement Overviews. It interacts with the Data Service.

- **Analytics Integration Component:** Facilitates Data Sync and Authentication for integration with an <<external>> Analytics system, providing Analytics Generation capabilities.
- **<> Database:** The central persistent storage for all system data, interacted with by various components via Data Stream and Storage Service.
- **<> Analytics System:** An external system that receives data from Analytics Integration for in-depth analysis.

The diagram effectively illustrates the modular nature of ProcureHub, showing how distinct services (e.g., Security, Notification, Data Service) are utilized by higher-level functional components (e.g., Procurement Request, Bidding and Proposal, Admin Dashboard) to deliver a comprehensive procurement solution. The explicit connections between components highlight the dependencies and information flow within the system boundary.



Picture 10: Component Diagram

6. External Interface Requirements

6.1 User Interface

ProcureHub will provide an intuitive and responsive web interface, optimized for both desktop use. The UI will be developed using **React.js** and **MaterialUI**, following modern UX/UI best practices.

Key interface components include:

- **Login/Register Screens** with role selection (Buyer, Seller)
- **Dashboards** for each user role:
 - Buyers: profile, procurement creation, procurement requests, analytics, contracts
 - Sellers: profile, list of available procurements, favorite requests, submitted bids, bid status, contracts, analytics
 - Admins: user management, contracts, platform statistics, reports
- **Auction Module UI** with real-time updates using WebSockets
- **Platform analytics** [Plausible.io](https://plausible.io) embedded
- **Notification System** (in-app + email)

6.2 Hardware Interface

As a cloud-based web application, ProcureHub does not directly interact with specific hardware components. However, the following hardware assumptions apply:

- Users must have access to an internet-enabled device (desktop, laptop, tablet).
- For optimal performance, devices should have:
 - Minimum 2GB RAM

- Dual-core processor or higher
- Stable internet connection (5 Mbps or higher recommended)

The platform may also support **PDF export and printing**, utilizing standard OS-connected printers.

6.3 Software Interface

ProcureHub integrates several software modules and third-party services to support robust auction and procurement workflows.

Backend APIs:

- **REST API:** Built with Express.js, enabling standard frontend-backend communication for procurement, bidding, authentication, and user management.
- **WebSocket API:** Provides real-time updates for auction events and bidding interactions.

Caching:

- **Redis:** Used to cache frequently requested data.

Third-party and Internal Services:

- **Authentication:** JWT-based custom auth for secure login and session handling.
- **Email Notifications:** Implemented using **Nodemailer**, supporting SMTP configuration.
- **Web Analytics:** **Plausible Analytics** provides privacy-friendly, cookie-less tracking of user activity for admin.

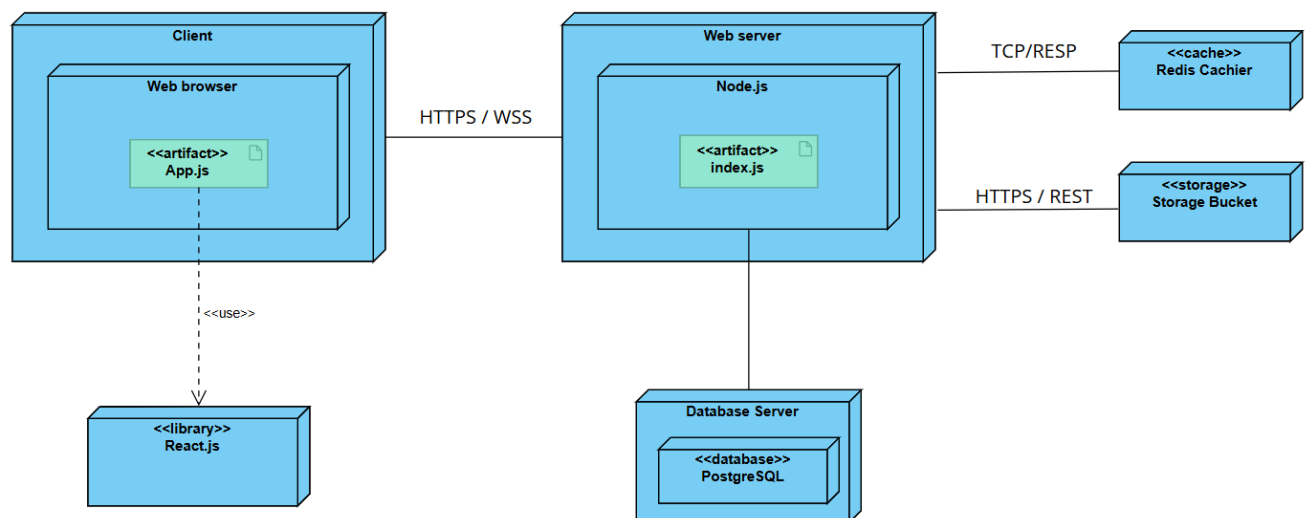
Database:

- **PostgreSQL**

6.4 Communication Interface and Deployment Diagram

Communication Interfaces:

- **HTTPS:** Secure RESTful communication between the frontend and backend.
- **WebSocket (WSS):** Used for bi-directional real-time communication, especially for live auctions.
- **SMTP (via Nodemailer):** Handles outgoing emails through a configured SMTP server for account verification, bidding alerts, and other notifications.
- **JWT:** Ensures secure token-based access control for authenticated services.
- **Redis Protocol:** Used internally for pub/sub and caching between backend services.



Picture 11: Deployment diagram

7. Non-Functional Requirements

7.1 System Usability

The system will be designed with usability as a priority to ensure that users can **easily learn, navigate, and perform key tasks efficiently**. The user interface will be **intuitive, consistent, and minimalistic**, focusing on clarity and reducing cognitive load.

Key usability principles include:

- Clear visual hierarchy and familiar UI patterns to reduce learning time.
- Responsive design for seamless usage across various devices and screen sizes.
- Use of legible fonts, sufficient contrast, and accessible components to support all users, including those with impairments.
- Minimal number of steps to complete common actions such as registration, bidding, or product listing.
- Immediate feedback on user actions to improve interaction transparency (e.g., form validation, real-time status updates).

The overall goal is to deliver a **smooth and frustration-free user experience** that encourages regular usage and increases user satisfaction.

7.2 System Security

All communications will be encrypted using HTTPS and WSS protocols.

JWT will be used for secure authentication and authorization.

Sensitive data such as passwords will be hashed using bcrypt before being stored.

Redis will be configured with appropriate access control and memory limits to prevent cache-based exploits.

7.3 System Availability

The system aims for **99.9% uptime**, ensured through hosting on scalable platforms such as Railway and redundant deployment environments.

Redis and PostgreSQL services will use managed or replicated setups (where applicable) to reduce risk of single points of failure.

7.4 System Maintainability

The codebase will follow modular design patterns (e.g., MVC on the backend, component-based structure on the frontend).

TypeScript (optional/for future) will be used to enforce code quality and consistency.

API documentation via Swagger/OpenAPI will simplify integration and maintenance.

CI pipelines will enforce unit tests and code linting before merges.

Git branching and pull request workflows will be used for collaborative development and code reviews.

7.5 System Scalability

Backend services will be stateless and horizontally scalable to support increased traffic and auction load.

Redis caching reduces database load by serving frequently accessed data quickly.

WebSocket server is optimized to handle thousands of concurrent connections via clustering or distributed pub/sub.

Cloud deployment on platforms like Railway supports vertical and horizontal scaling based on demand.

7.6 System Performance

Average page load time will be under 2 seconds on broadband connections.

Real-time auction updates will be delivered with latency under 500ms using WebSockets.

Redis will be used to cache hot data, reducing database response time.

Backend API endpoints will respond within 300ms under normal load.

8. FURPS+ Model

8.1 Functionality

User onboarding & access control: User registration, secure login (JWT), and profile setup for all user roles (seller, buyer, admin).

User management (admin): CRUD operations on user profiles, managing access control (approve, suspend, delete).

Procurement requests: Buyers can create procurement requests with detailed specifications, set bid evaluation criteria, and monitor active requests.

Bidding & auctions (seller): Sellers can browse and filter procurement requests, submit bid proposals, upload supporting documents, participate in live auctions with real-time notifications and bidding.

Auction management (buyer/admin): Define auction rules, oversee live auctions, and enable automatic auction extensions when needed.

Contract management: View, accept, or dispute contract terms; generate and issue contracts; track contract lifecycle.

Dispute resolution: File disputes and complaints, report suspicious activities, and resolve conflicts fairly.

Analytics & reporting: Provide insights and reports tailored to sellers, buyers, and admins about activities, performance, and platform usage.

Notifications: Send email and in-platform notifications about bids, auctions, contracts, and other key events.

8.2 Usability

Intuitive UI: Clear and organized workflows for registration, login, browsing requests, bidding, auction participation, and contract management.

Responsive design

User tools: Features like favorites saving, filtering options, activity history, and role-specific analytics views.

Clear messaging: Informative error messages, confirmations, and status updates to guide users.

8.3 Reliability

Security: Secure authentication with JWT, role-based access control (RBAC), and session management.

Error handling: Robust data validation and error feedback mechanisms to users.

Backup and audit: Regular data backups and audit logs to track user and admin actions.

Stable operation: Fault-tolerant system with minimal downtime, especially during live auctions and bid submissions.

8.4 Performance

Scalability: Support for concurrent users, especially during peak auction times.

Low latency: Real-time updates on auctions and bids with minimal delay.

Efficient database: Optimized searching and filtering of procurement requests and bids.

Load balancing: Even distribution of server load to maintain responsiveness under high traffic.

8.5 Supportability

Modular architecture: Clear separation of features by teams and components for easier maintenance and upgrades.

Documentation: User guides, and technical specifications.

Monitoring and logging: Continuous system monitoring and detailed event logs for diagnostics and support.

Deployment and updates: Ability to deploy patches and new versions with minimal or no downtime.

9. Implementation Requirements

9.1 Interface Constraints

User Interface: The platform must provide intuitive web interfaces accessible via modern browsers (Chrome, Firefox, Edge, Safari)

API Interface: The backend will expose RESTful APIs for all core functionalities (user management, procurement requests, bidding, auctions, contracts, analytics) with JSON data format.

Authentication: Use JWT tokens for secure API authentication and session management across frontend and backend.

Third-party Integration: Interfaces must support integration with email services

9.2 Physical Requirements

Hosting environment: The application will be deployed on cloud servers supporting Node.js backend and a suitable database system (PostgreSQL).

Network: Stable internet connectivity is required to support real-time auction features with low latency.

Storage: Adequate storage for user data, procurement documents, bids, contracts, and logs. Secure file storage is required for uploaded documents (PDFs, images).

Hardware: The system should be compatible with standard user devices (PCs, laptops, tablets, smartphones) without additional hardware dependencies.

9.3 Design Constraints

Technology stack: Frontend built with React (or similar modern framework), backend using Node.js and Express.js, with PostgreSQL database.

Security: Implementation must follow best practices for data protection, including encryption of sensitive data, secure authentication, and authorization mechanisms.

Modularity: The system must be designed to separate concerns between Seller, Buyer and Admin Features

Scalability: The architecture should allow horizontal scaling to handle increased load during peak usage, such as live auctions.

Compliance: The system must adhere to relevant legal and regulatory standards for data privacy and procurement practices (e.g., GDPR if applicable).

Maintainability: Codebase must be clean, well-documented, and structured to support ongoing enhancements, bug fixes, and feature expansions.